

Examen Final de GRAU-IA

(16 de enero de 2024)

Duración: 2 horas 30 minutos

1. (5 puntos) Una importante marca de móviles y tablets desea ampliar los servicios que da a sus clientes creando su propia tienda de contenidos multimedia (a la que llamará *Media Store*). La parte innovadora es que quiere que la tienda genere recomendaciones personalizadas a los usuarios de sus móviles y tablets sobre que contenidos podría comprar a partir de las compras que ya ha realizado y la información que ha recolectado de los usuarios que han hecho compras similares a las suyas. Los productos son el contenido principal que se vende en la *Media Store* y pueden ser libros electrónicos, películas y música (canciones individuales). También se ofrecen packs de productos (cada pack tiene un título y un precio), que pueden ser todos del mismo tipo (por ejemplo, un álbum entero de un cantante o grupo musical es un tipo especial de pack con las canciones de ese álbum) o una mezcla de diferentes tipos (por ejemplo, un pack compuesto por una película y el libro en el que se basa dicha película).

Se ha decidido utilizar un sistema basado en el conocimiento capaz de determinar el perfil al que pertenece un usuario usando las características de todas las compras realizadas de cada tipo de producto. De todas formas, para poder refinar más las recomendaciones guardamos también la cantidad de productos de cada tipo que ha comprado el usuario a lo largo del tiempo.

De los libros electrónicos se guarda información como el título, el autor, la fecha de publicación y el precio. Los libros están categorizados en Categorías y sub-categorías: Didácticos (Libro de Cocina, Libro de Autoayuda, Ensayo, Biografía, Libro Científico-técnico), Lírico (Poesía) y Narrativo (Novela, Cuento y Cómic). De las películas se guarda información como el título, el director, la fecha de publicación, su duración, el número de premios recibidos y el precio. Estas están categorizadas según su género en acción, comedia, drama, documental, ciencia-ficción, animación, cine clásico, etc. En el caso de que la película esté basada en una novela o en un comic dentro del catálogo de la *Media Store* esto también se guarda. De las canciones se guarda información como el título, intérprete, fecha de publicación, duración y precio. Estas están categorizadas por su género musical en jazz, pop-rock, country, techno, urbano, música étnica, música clásica, música latina, etc. Si una canción forma parte de una banda sonora que pertenece a una película del catálogo de la *Media Store* esto también se guarda.

Para simplificar las cosas se ha determinado un conjunto de características que permiten categorizar mejor a los usuarios de la *Media Store* y poder guiar el proceso de recomendación:

- el **tipo de usuario**, que puede ser *ocasional*, *un-poco-de-todo*, *melómano*, *cinéfilo* o *lector*. Los usuarios *ocasionales* son los que han tenido menos de diez compras en cada tipo de producto, y a estos solo se les puede recomendar los productos más vendidos del catálogo de la *Media Store*. Los usuarios *un-poco-de-todo* son los que no tienen unas preferencias definidas y compran productos de todas las categorías, a estos se les pueden recomendar solo los productos más vendidos del catálogo dada su indefinición. Los usuarios de tipo *cinéfilo* son aquellos usuarios que compran mayoritariamente cine, y se les recomendará según su perfil de cine (ver a continuación). Los usuarios de tipo *melómano* son aquellos usuarios que compran mayoritariamente canciones, y sus recomendaciones se construirán a partir de su perfil musical (ver a continuación). Finalmente los usuarios de tipo *lector* son aquellos usuarios que compran mayoritariamente libros electrónicos, y a los que se les recomendará según su perfil lector (ver a continuación).
- el **perfil de cine**, que puede ser *nulo* (no compra cine), *indefinido* (compra cine de cualquier género), o bien alguno de los géneros (*acción*, *comedia*, *drama*, *documental*, *ciencia-ficción*, *animación*, *cine clásico*, etc.) si sus compras son mayoritariamente de ese género. En el caso de perfil de cine *nulo* o *indeterminado* las recomendaciones se construirán según el tipo de usuario (que será *ocasional* o *un poco de todo*, ver arriba). En el caso de perfil de cine *acción* se les recomendarán las películas más vendidas del género acción. En el caso de perfil de cine *drama* se le recomendarán las películas más premiadas del género drama. Para el resto de valores del perfil de cine (*comedia*, *documental*, *ciencia-ficción*, *animación*, etc.) se le recomendarán las novedades de cine del catálogo relacionadas con ese género.
- el **perfil musical**, que puede ser *nulo* (no compra canciones), *indefinido* (compra canciones de cualquier género), o bien alguno de los géneros (*jazz*, *pop-rock*, *country*, *techno*, *urbano*, *música étnica*, *música clásica*, *bandas sonoras*, etc.) si sus compras son mayoritariamente de ese género. En el caso de perfil musical *nulo* o *indeterminado* las recomendaciones se construirán según el tipo de usuario

(que será *ocasional* o *un poco de todo*, ver arriba). En el caso de perfil de música *jazz* o *música clásica* se les recomendarán las canciones más vendidas entre los melómanos de esos géneros. Para el resto de valores del perfil musical (*pop-rock*, *country*, *techno*, *urbano*, etc.) se le recomendarán las novedades de canciones del catálogo relacionadas con ese género.

- el **perfil lector**, que puede ser *nulo* (no compra libros electrónicos), *indefinido* (compra libros electrónicos de cualquier subcategoría), o bien alguna de las subcategorías (*Libro de Cocina*, *Libro de Autoayuda*, *Ensayo*, *Biografía*, *Libro Científico-técnico*, *Poesía*, *Novela*, *Cuento*, *Cómic*, etc.) si sus compras son mayoritariamente de esa temática. En el caso de perfil lector *nulo* o *indefinido* las recomendaciones se construirán según el tipo de usuario (que será *ocasional* o *un poco de todo*, ver arriba). En el caso de perfil lector *Novela* o *Ensayo* se les recomendarán los libros más vendidos entre los lectores de esas subcategorías. Para el resto de valores del perfil lector se le recomendarán las novedades de libros del catálogo relacionadas a esa subcategoría.
- el **gasto mensual** en productos de la *Media Store*, que puede ser *bajo* (por debajo de 20 euros/mes en el último año), *normal* (entre 21 euros/mes y 100 euros/mes en el último año) o bien *elevado* (por encima de 100 euros/mes en el último año). Para los usuarios con gasto mensual *bajo* se construirán las recomendaciones a partir de las ofertas del mes. Para los usuarios con gasto mensual *normal* se construirán las recomendaciones a partir de las características listadas arriba. Para los usuarios con gasto mensual *elevado* se añadirán a las recomendaciones packs de productos relacionados con su perfil.

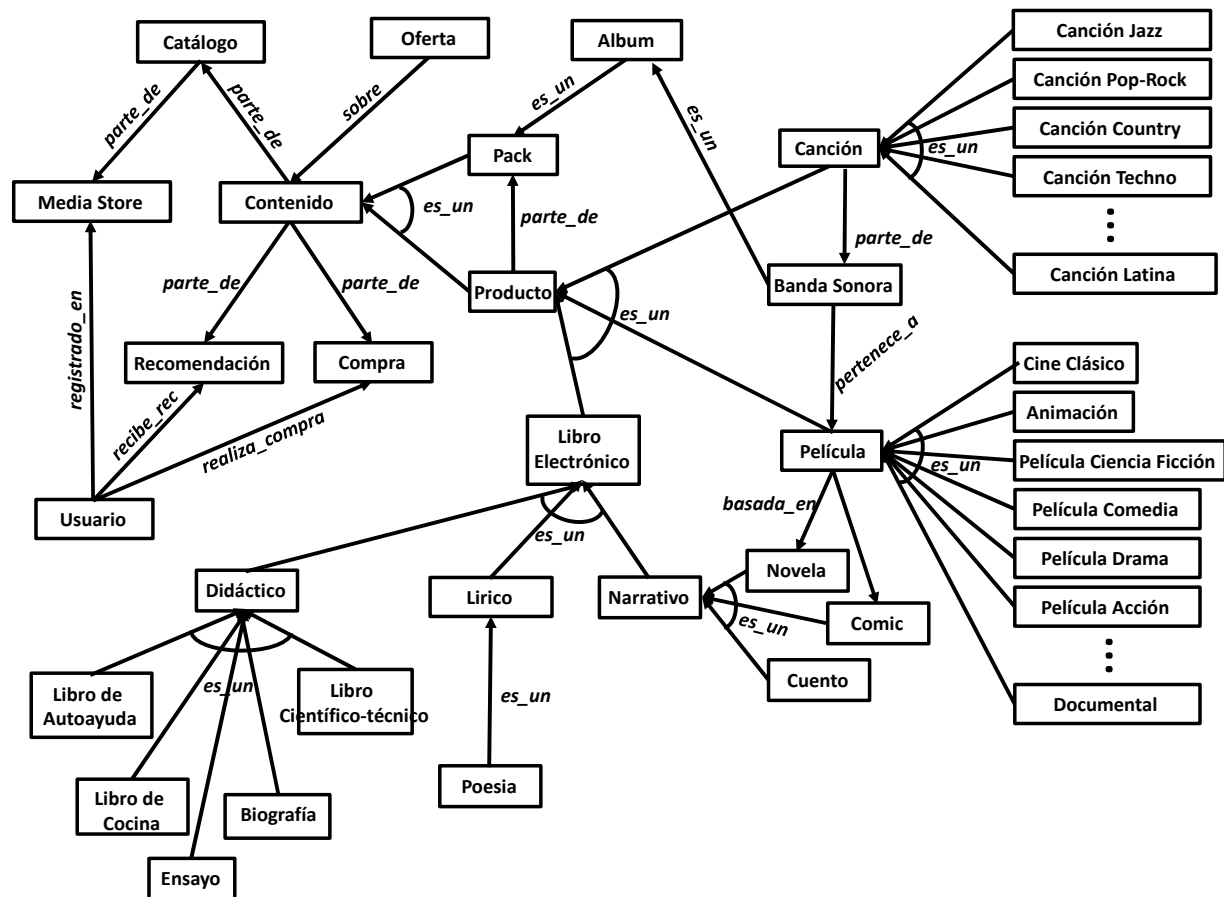
Evidentemente un usuario no puede ser clasificado siempre en una única categoría, dadas sus últimas compras puede que cumpla características de varios tipos de compradores, por lo que la recomendación final debería ser una combinación de las recomendaciones adecuadas (los elementos subrayados arriba) para cada una de las categorías de comprador a las que pertenezca.

El objetivo es construir un sistema capaz de generar una recomendación de productos del catálogo de la *Media Store* adecuados a cada usuario del dispositivo (móvil o tablet).

- a) (2,5 puntos) Diseña la ontología del dominio descrito, incluyendo todos los conceptos que aparecen en la descripción e identificando los atributos más relevantes. Lista que conceptos forman parte de los datos de entrada del problema y que conceptos forman parte de la solución. Justifica las decisiones de diseño que has realizado. En el caso de las subclases de una clase, justifica con un texto breve la necesidad de crear dichas subclases. Para cada concepto de la ontología lista todos sus atributos (para cada atributo se ha de indicar su nombre y su tipo, y en el caso de las enumeraciones, sus valores posibles). [Nota: tened en cuenta que la ontología puede necesitar modificaciones para adaptarla al apartado siguiente.]

En este problema la ontología ha de incorporar todos los conceptos que forman parte de la entrada del problema (la información de todos los contenidos del catálogo de la *Media Store* y las compras realizadas por los usuarios) y también ha de dar soporte a la salida del SBC (recomendaciones de contenidos al usuario).

De la ontología resultante cabe remarcar que existen varios conceptos que comparten atributos y/o relaciones, y que esto ha sido algo que se ha tenido muy en cuenta a la hora de crear super-clases con las características comunes, o para decidir entre crear subclases o solo añadir un atributo **tipo**. La regla general es que hemos creado subclases cuando una o varias tienen atributos diferentes (por ejemplo, las subclases de **Producto**) y/o relaciones diferentes (por ejemplo, en el caso de **Libro electrónico** las subclases **Novela** y **Comic** tienen una relación con **Película** que sus clases hermanas no tienen, lo cual justifica que despleguemos la clase **Narrativo** y todas sus subclases). Y debido a esto se han incluido la otras clases y subclases de **Libro Electrónico**, que a pesar de que no tienen ni atributos ni relaciones diferenciadas se han añadido siguiendo el heurístico del diseño de ontologías que recomienda que ramas hermanas de la taxonomía tengan un nivel de granularidad similar. De hecho esto nos ha hecho también desplegar las subclases de **Película** y **Canción**.



Una decisión importante ha sido añadir la clase **Contenido** para agrupar los atributos (título, precio) y multiples relaciones que comparten las clases **Producto** y **Pack**.

Otra decisión importante ha sido el añadir el concepto **Compra** en vez de una relación directa *ha_comprado* entre **Usuario** y **Contenido**, ya que en este último caso no podríamos asociar la fecha de compra. Si se quisiera hacer la ontología aún más general se podrían convertir los atributos **autor**, **director** e **intérprete** en conceptos que heredan de **Persona**, y añadir el concepto **Premio** asociado al de **Película**.

Atributos: a continuación se listan los atributos mínimos para representar la información mencionada explícitamente en el enunciado y la necesaria para el apartado b) del problema. Hay otros atributos que se deberían añadir (como el número de referencia de cada producto o la ficha completa de cada película) si quisiéramos una ontología lo más general y reutilizable posible.

- **Usuario:** nombre (string), num_peliculas (entero), num_canciones (entero), num_libros (entero), compra_mes-1 (euros), compra_mes-2 (euros), compra_mes-3 (euros), compra_mes-4 (euros), compra_mes-5 (euros), compra_mes-6 (euros), compra_mes-7 (euros), compra_mes-8 (euros), compra_mes-9 (euros), compra_mes-10 (euros), compra_mes-11 (euros), compra_mes-12 (euros), *Tipo_usuario* (enumeración: {ocasional, un_poco_de_todo, melómano, cinéfilo, lector}), *Gasto_mensual* (enumeración: {bajo, normal, elevado}), *Perfil_cine* (enumeración: {nulo, indefinido, acción, comedia, drama, documental, ciencia-ficción, animación, cine_clásico, etc.}), *Perfil_lector* (enumeración: {nulo, indefinido, libro_de_cocina, libro_de_autoayuda, Ensayo, Biografía, libro_científico_técnico, poesia, novela, cuento, cómic, etc.}), *Perfil_musical* (enumeración: {nulo, indefinido, jazz, pop-rock, country, techno, urbano, música_étnica, música_clásica, banda_sonora, etc.});
- **Compra:** fecha_compra (fecha);
- **Recomendación:** *RECOMENDACIÓN* (enumeración: {ofertas_del_mes, productos_más_vendidos, pack_por_perfil, películas_acción_más_vendidas, películas_drama_más_vendidos});

más_premiadas, novedades_cine_por_género, canciones_más_vendidas_por_género, novedades_canciones_por_género, libros_clásicos_más_vendidos, novedades_libros_por_tema});

- **Contenido:** título (string), precio (euros);
- **Producto:** fecha_publicación (fecha);
- **Canción:** intérprete (string), duración (segundos);
- **Película:** director (string), duración (segundos), num_premios (entero);
- **Libro Electrónico:** autor (string);

Cabe destacar que se han añadido los atributos `Usuario.compra_mes-1`, `Usuario.compra_mes-2` .. `Usuario.compra_mes-12` para dar soporte al pre-procesado de datos antes de la abstracción (ver apartado b). Como se puede ver, hemos representado también las características abstractas del problema y las de la solución abstracta (son los que tienen el nombre en *Cursiva* o *CURSIVA*, respectivamente). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

- b) (2,5 puntos) El problema descrito es un problema de análisis. Explica cómo lo resolverías usando clasificación heurística. Justifica si el tipo de solución que se pide requiere de refinamiento o no. Lista las variables que forman parte del Problema Abstracto (nombre y valores posibles). Lista las variables que forman parte de la Solución Abstracta (nombre y valores posibles). Da al menos 3 ejemplos de reglas (suficientemente variadas, utilizando diferentes conceptos) para cada una de las fases de esta metodología, usando los conceptos de la ontología desarrollada en el apartado anterior. Para las reglas podéis usar la notación de alto nivel que usamos en los ejercicios de problemas (*si ... entonces* ...) o una notación más próxima a la del lenguaje CLIPS (siempre que queden claro los elementos del antecedente de la regla y del consecuente).

Para resolverlo mediante clasificación heurística debemos identificar en el problema las diferentes fases y elementos de esta metodología. En este caso hay dos opciones posibles. Si consideramos que las recomendaciones que ha de dar el sistema (ofertas del mes, productos más vendidos, películas de acción más vendidas, películas de drama más vendidas, novedades de cine de un cierto género, etc.) son soluciones pre-calculadas (cada día, cada semana) que no requieren de información específica de las compras del usuario entonces se puede considerar que la salida del sistema es una solución abstracta. La alternativa es considerar que, incluso si estas soluciones han sido precalculadas, necesitamos información concreta de las compras del usuario para acabar de ajustar la lista de contenidos recomendados (por ejemplo, para eliminar los que el usuario ya ha comprado) y entonces la salida del sistema es una solución concreta. Es esta segunda opción la que vamos a describir a continuación.

Una vez sabemos cuantas fases requiere nuestra solución ya podemos enunciar la solución completa. El primer elemento son los problemas concretos que hay que tratar. En este dominio los problemas concretos están definidos por toda la información detallada sobre el catálogo de la *Media Store* y las compras realizadas por el usuario, tal y como los hemos identificado en la ontología. En este caso hace falta un preproceso de los datos, ya que en algunas reglas de abstracción se necesita pre-calcular algunos valores (como el gasto en euros realizado en cada uno de los últimos 12 meses del año, que se almacenará en `Usuario.compra_mes-1`, `Usuario.compra_mes-2` .. `Usuario.compra_mes-12`) o el número de compras realizadas de cada tipo de producto (`Usuario.num_peliculas`, `Usuario.num_canciones` y `Usuario.num_libros`). Suponemos que tenemos métodos para realizar todos estos cálculos a partir de los datos que introduce el usuario, y estos quedan almacenados en la ontología mediante los atributos extra que añadimos para tal efecto.

El segundo elemento son los problemas abstractos, estos estarán definidos a partir de las cinco características que menciona el enunciado (*Tipo_usuario*, *Gasto_mensual*, *Perfil_cine*, *Perfil_lector* y *Perfil_musical*). Para conectar los problemas concretos con los abstractos necesitamos definir las reglas de abstracción de datos, por ejemplo:

- si Usuario.num_libros < 10 y Usuario.num_películas < 10
y Usuario.num_canciones < 10 entonces Usuario.Tipo_usuario=ocasional;
 - si Usuario.num_libros > 10 y Usuario.num_libros > (2 * Usuario.num_películas +
Usuario.num_canciones) entonces Usuario.Tipo_usuario=lector;
 - si Usuario.num_películas > 10 y Usuario.num_películas > (2 * Usuario.num_libros +
Usuario.num_canciones) entonces Usuario.Tipo_usuario=cinéfilo;
 - si Usuario.num_películas = 0 entonces Usuario.Perfil_cine=nulo;
 - si Usuario.num_canciones = 0 entonces Usuario.Perfil_musical=nulo;
 - si $\frac{\text{cardinalidad}([\text{Usuario.realiza_compra.parte_de}] \text{ es_un Canción_Jazz})}{\text{Usuario.num_canciones}} > 0.4$
entonces Perfil_musical=Jazz;
 - si $\frac{\text{cardinalidad}([\text{Usuario.realiza_compra.parte_de}] \text{ es_un Poesia})}{\text{Usuario.num_libros}} > 0.35$
entonces Perfil_lector=Poesia;
 - si entropia(Usuario.realiza_compra.parte_de, Libro_electrónico) > 0.7
entonces Perfil_lector=indefinido;
- .
- si compra_mes-1 < 20euros o compra_mes-2 < 20euros o compra_mes-3 < 20euros o
compra_mes-4 < 20euros o compra_mes-5 < 20euros o compra_mes-6 < 20euros o
compra_mes-7 < 20euros o compra_mes-8 < 20euros o compra_mes-9 < 20euros o
compra_mes-10 < 20euros o compra_mes-11 < 20euros o, compra_mes-12 < 20euros
entonces Usuario.Gasto_mensual=bajo;

El tercer elemento son las soluciones abstractas. En este caso tenemos la lista de recomendaciones subrayadas en el enunciado, convertidas en una variable cualitativa que corresponde con el atributo *RECOMENDACIÓN* del concepto *Recomendación*: {ofertas_del_mes, productos_más_vendidos, pack_por_perfil, películas_acción_más_vendidas, películas_drama_más_premiadas, novedades_cine_por_género, canciones_más_vendidas_por_género, novedades_canciones_por_género, libros_clásicos_más_vendidos, novedades_libros_por_tema}).

Para ligar los problemas abstractos con las soluciones abstractas necesitaremos reglas de asociación heurística, como por ejemplo:

- si Usuario.Gasto_mensual=bajo
entonces Usuario.recibe_rec.RECOMENDACIÓN=ofertas_del_mes;
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=(ocasional o un_poco_de_todo)
entonces Usuario.recibe_rec.RECOMENDACIÓN=productos_más_vendidos;
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=melómano y
Usuario.Perfil_musical=Jazz
entonces Recomendación.RECOMENDACIÓN=canciones_más_vendidas_por_género(Jazz);
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=melómano y
Usuario.Perfil_musical=Música_clásica entonces Usuario.recibe_rec.RECOMENDACIÓN=
canciones_más_vendidas_por_género(Música_clásica);
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=melómano y
Usuario.Perfil_musical≠Jazz y Usuario.Perfil_musical≠Música_clásica
entonces Usuario.recibe_rec.RECOMENDACIÓN=
novedades_canciones_por_género(Usuario.Perfil_musical);
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=cinéfilo y
Usuario.Perfil_cine=acción
entonces Usuario.recibe_rec.RECOMENDACIÓN=películas_acción_más_vendidas;
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=cinéfilo y
Usuario.Perfil_cine=drama
entonces Usuario.recibe_rec.RECOMENDACIÓN=películas_drama_más_premiadas;
- si Usuario.Gasto_mensual≠bajo y Usuario.Tipo_usuario=lector y
Usuario.Perfil_lector=(novela o ensayo
entonces Usuario.recibe_rec.RECOMENDACIÓN=
libros_más_vendidos(Usuario.Perfil_lector);

El cuarto y último elemento son las soluciones concretas. En este caso corresponde a la generación de las salidas del sistema teniendo en cuenta el resultado de la fase anterior y teniendo en cuenta datos concretos del problema para poder generar las salidas descritas en el enunciado. En este caso necesitamos solo una reglas de producción para el refinamiento. Suponemos la existencia de un método `generar_lista_recomendaciones` que dada la recomendación obtenida y las compras del usuario genera la lista de contenidos que se le mostrará en la pantalla de su dispositivo.

- si Usuario.recibe_rec.RECOMENDACIÓN=ofertas_del_mes
 entonces generar_lista_ofertas_mes (Usuario.compra_mes-1);
- si Usuario.recibe_rec.RECOMENDACIÓN= canciones_más_vendidas_por_género(genero)
 entonces generar_lista_canciones_por_género(genero, Usuario);
- si Usuario.recibe_rec.RECOMENDACIÓN= libros_más_vendidos(perfil)
 entonces generar_libros_top_ventas_por_perfil(perfil, Usuario);

2. (5 puntos) El mago Rincewind debe crear un hechizo colocando N nexos arcanos en una cuadrícula de $N \times N$ posiciones. Para que el hechizo funcione, es necesario que los N nexos estén en armonía, lo cuál requiere que no compartan fila, columna ni diagonales.

a) (4 puntos) Describe el dominio (incluyendo predicados, acciones, etc.) usando PDDL. Se valorará especialmente que la descripción sea eficiente a nivel temporal.

Como marca el enunciado, se ha valorado especialmente la eficiencia temporal. Listamos a continuación cosas que se han valorado:

- Uso de predicados guía: forzar a poner la reina de cada fila en un orden determinado ($f_1, f_2 \dots$).
- Uso correcto de tipos, con tipos diferentes para filas, columnas y diagonales. El uso de dos tipos diferentes para cada dirección de diagonal se ha considerado especialmente importante, ya que mejora la eficiencia (más fácil hacer match) y en muchos casos no comprobarlo permitía hacer 'colocar(f_1, c_1, d_5, d_5)' repitiendo la diagonal y rompiendo el código.
- Evitar el uso de forall o exists por completo. En caso de usarse, se ha penalizado especialmente su uso en la precondition, y no tanto en el efecto.
- No comprobar que la elección de fila y columna no corresponda con la diagonal escogida se ha penalizado severamente (p.ej, f_1, c_1, d_1 no sería correcto si d_1 no pasa por c_1, f_1).
- No comprobar ambas diagonales se ha penalizado severamente.
- La ordenación de los predicados en la precondition de la acción también se ha valorado (mejor los más restrictivos primero, lo que implica comprobar diagonales-fila-columna primero si bien depende de los predicados usados caso por caso).
- Crear una acción de descolocar reina se ha penalizado severamente, ya que incrementa el espacio de búsqueda de forma exponencial.

A continuación mostramos una solución subóptima pero suficiente para máxima puntuación. Hay una versión muchísimo más sencilla y eficiente: marcar líneas como *threatened* en lugar de posiciones elimina la necesidad de hacer forall o exists. Soluciones incluyendo exists o forall en la precondition han recibido una penalización de 1 punto.

```
(define (domain RincewindNNexus)

  (:requirements :strips :adl :typing)

  (:types position line - object
    row column diag_left diag_right - line
  )

  (:predicates
```

```

(threatened ?p - position)
(part_of ?p-position ?l-line)
(nexus_placed ?p-position)

(current_row ?l -row)
(next_row ?l1 ?l2 -row)
)

(:action place
:parameters (?p - position ?r -row ?col -column ?diag1 -diag_left
             ?diag2 - diag_right ?next_row - row)
:precondition (and
  (current_row ?r) (part_of ?p ?r) (part_of ?p ?col)
  (part_of ?p ?diag1) (part_of ?p ?diag2)
  (not (threatened ?p))
  (next_row ?r ?next_row)
)
:effect ( and
  (nexus_placed ?p-position)
  (forall (?p2 - position)
    (and
      (when (part_of ?p2 ?r)(threatened ?p2))
      (when (part_of ?p2 ?col)(threatened ?p2))
      (when (part_of ?p2 ?diag1)(threatened ?p2))
      (when (part_of ?p2 ?diag2)(threatened ?p2))
    )
  )
  (not (current_row ?r))
  (current_row ?next_row)
)
) )

```

Solución óptima:

```

(define (domain RincewindNNexus)

  (:requirements :strips :adl :typing)

  (:types line - object ; Uso line para no utilizar predicados con object por
                        ; preferencia personal
        row column diag_left diag_right - line
  )

  (:predicates
    (threatened ?l - line)
    (convergence ?r-row ?c-column ?d1-diag_left ?d2 - diag_right)
    (current_row ?l -row)
    (next_row ?l1 ?l2 -row)
  )

  (:action place
    :parameters (?r -row ?col -column ?diag1 -diag_left ?diag2 - diag_right
                 ?next_row - row)
  )

```

```


```

:precondition (and
 (current_row ?r) (convergence ?r ?c ?diag1 ?diag2)
 (not (threatened ?c))
 (not (threatened ?diag1))
 (not (threatened ?diag2))
 (next_row ?r ?next_row)
)
:effect (and
 (threatened ?r)
 (threatened ?c)
 (threatened ?diag1)
 (threatened ?diag2)
 (not (current_row ?r))
 (current_row ?next_row)
)
))

```


```

- b) (1 puntos) Representa el fichero de problema para la colocación de 5 nexos en PDDL acorde con vuestra propuesta de dominio del apartado anterior.

```

(define (problem 5Nexus)
  (:domain RincewindNNexus)
  (:objects
    P11 P12 ... P15 P21 ... P55 - position
    R1 ... R5 RF - row
    C1 ... C5 - column
    DL1 ... DL9 - diag_left
    DR1 ... DR9 - diag_right
  )
  (:init
    (current_row R1)
    (next_row R1 R2) (next_row R2 R3) ... (next_row R5 RF)

    (part_of P11 R1) (part_of P11 C1) (part_of P11 DL1) (part_of P11 DR5)
    (part_of P12 R1) (part_of P12 C2) ....
    ; part of row iff row=RX and position=PX
    ; part of col iff col=CY and position=PX
    ; part of diag_left iff diag=DL(X+Y-1) and position=PX
    ; part of diag_right iff diag=DR(Y-X+N) and position=PX

    ; En la 2ª solución sustituir predicados part_of por convergence siguiendo
    ; las mismas indicaciones:
    ; (convergence R1 C1 DL1 DR5) (convergence R1 C2 DL2 DR4)

  )
  (:goal (current_row RF))
)

```